

Выполнила Кузнецова Валерия, гр. 3825М1ФИИ1

Week01

1. Базовые понятия RL

Обучение с подкреплением (Reinforcement Learning, RL) – это метод машинного обучения, при котором агент обучается принимать оптимальные решения в процессе взаимодействия со средой. В отличие от обучения с учителем, здесь нет эталонных ответов: агент получает обратную связь в виде сигналов награды.

Основные компоненты процесса (формализм MDP – Марковского процесса принятия решений):

- Агент (Agent) – система, принимающая решения и выполняющая действия.
- Среда (Environment) – внешний мир, с которым взаимодействует агент.
- Состояние (State, s) – описание текущей ситуации в среде. Множество всех состояний обозначается $S, s \in S$.
- Действие (Action, a) – выбор, который делает агент. Множество всех действий обозначается $A, a \in A$.
- Награда (Reward, r) – скалярный сигнал обратной связи от среды, показывающий качество действия, $r \in \mathbb{R}$.
- Политика (Policy, π) – стратегия поведения агента, определяющая, какое действие выбрать в каждом состоянии. Политика $\pi(a|s) = P(a|s)$ определяет вероятность выбора действия a в состоянии s .
- Марковское предположение: $P(s_t + 1 | s_t, a_t, s_{t-1}, a_{t-1}) = P(s_{t+1} | s_t, a_t)$ означает, что вероятность перехода в следующее состояние зависит только от текущего состояния и действия, а не от всей предыстории.

Суммарная награда за сессию: $R = \sum_t r_t$. Цель агента – найти такую стратегию, которая максимизирует суммарную ожидаемую награду за сессию: $\pi(a|s): E_\pi[R] \rightarrow \max$.

2. Метод кросс-энтропии (табличный + нейронная сеть)

Метод кросс-энтропии (Cross-Entropy Method, CEM) – это алгоритм обучения с подкреплением, основанный на эволюционном подходе. Это простой, но эффективный метод, особенно для задач с непрерывным пространством действий.

Основная идея: итеративно улучшать политику, отбирая лучшие траектории (элитные сессии) и обучаясь на них.

Табличный метод кросс-энтропии применяется, когда пространство состояний и действий невелико и может быть описано матрицей. Алгоритм такой:

1. Инициализировать политику как матрицу вероятностей: $\pi(a|s) = A_{s,a}$.
2. Сгенерировать N траекторий (игр) с текущей политикой.
3. Выбрать M лучших сессий (элитные) по суммарной награде: $Elite = [(s_0, a_0), (s_1, a_1), (s_2, a_2), \dots, (s_k, a_k)]$.
4. Обновить $\pi(a|s) = \frac{count(a,s)}{count(s)}$, где $count$ — количество пар (s, a) в элитных сессиях.
5. Повторять шаги 2-4 до сходимости.

Метод с нейронной сетью. Для больших пространств состояний вместо таблицы используется нейронная сеть, аппроксимирующая $\pi(a|s)$. Шаги следующие:

1. Инициализировать веса W нейронной сети случайным образом.
2. Сгенерировать N сессий с текущей политикой.
3. Выбрать M элитных траекторий: $Elite = [(s_0, a_0), (s_1, a_1), (s_2, a_2), \dots, (s_k, a_k)]$.
4. Обучить нейросеть максимизировать логарифм правдоподобия действий из элитных сессий: $\pi = \sum_{s_i, a_i \in Elite} \log \pi(a_i | s_i)$. В коде это выглядит как стандартное обучение классификатора: `nn.fit(elite_states, elite_actions)`. Также обновляются веса W : $W_{i+1} = W_i + \alpha \nabla [\sum_{s_i, a_i \in Elite} \log \pi_{W_i}(a_i | s_i)]$.
5. Повторять до сходимости.

Для СМЕ существует несколько полезных трюков:

- Можно запоминать элитные сессии за последние 3–5 итераций, чтобы обучение было стабильнее.
- Регуляризация энтропией помогает агенту не заикливаться на одном действии слишком рано (предотвращает преждевременную сходимость).
- Процесс сбора данных легко ускорить, запустив его одновременно на нескольких ядрах процессора.

Week02

1. $V(s)$, $Q(s, a)$ – что это такое?

$V(s)$ и $Q(s, a)$ – это функции ценности, которые позволяют оценить, насколько хорошим является состояние или пара состояние-действие при следовании определённой политике. Оба понятия базируются на доходе (G_t) – сумме дисконтированных наград, которые агент планирует получить в будущем.

Функция ценности состояния $V_\pi(s)$ (State-value function):

$V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ – это математическое ожидание дохода, который агент получит, начиная из состояния s и далее следуя стратегии π . Она показывает «полезность» нахождения в данном состоянии при соблюдении текущей стратегии. При расчете $V(s)$ учитывается стохастичность (случайность) как самой стратегии (какие действия выберет агент), так и среды (в какие состояния агент попадет после действия).

Функция ценности действия $Q_\pi(s, a)$ (Action-value function):

$Q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ – ожидаемый доход при условии, что агент находится в состоянии s , совершает конкретное действие a , а затем продолжает следовать стратегии π . Она оценивает эффект от конкретного первого шага в текущей ситуации. В отличие от $V(s)$, при расчете $Q(s, a)$ на первом шаге нет неопределенности стратегии, так как действие a уже выбрано агентом принудительно.

Связь между ними: ценность состояния можно представить как сумму ценностей всех возможных действий в этом состоянии, умноженных на вероятность их выбора согласно стратегии: $V_\pi(s) = \sum_a Q_\pi(s, a) * \pi(a|s)$.

2. Метод Policy Iteration

Policy Iteration – итеративный алгоритм поиска оптимальной политики, состоящий из двух чередующихся этапов: оценки политики (Policy Evaluation) и улучшения политики (Policy Improvement). Он является частным случаем концепции обобщенной итерации стратегии (GPI).

Алгоритм Policy Iteration:

1. В начале работы функции ценности $V(s)$ и политика π инициализируются произвольно (например, нулями и случайными действиями).
2. Повторять следующие этапы, пока политика не станет изменяться ($\pi' = \pi$):

1) Оценка политики (Policy Evaluation).

На этом этапе агент вычисляет точную ценность текущей политики. Он многократно применяет уравнение Беллмана для ожидания, обновляя значения для всех состояний до тех пор, пока они не перестанут существенно меняться (сойдутся):

$$V(s) = \sum_{s',r} p(s',r | s, \pi(s)) [r + \gamma V(s')]$$

2) Улучшение политики (Policy Improvement).

После того, как ценность текущей политики известна, агент обновляет саму стратегию. В каждом состоянии он выбирает новое действие, которое является «жадным» по отношению к вычисленной функции ценности (то есть действие, максимизирующее $Q(s, a)$):

$$\pi'(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r | s, a) [r + \gamma V(s')]$$

Алгоритм сходится к оптимальной политике за конечное число итераций для конечных MDP. По сравнению с методом Value Iteration, метод Policy Iteration обычно требует меньше итераций для полной сходимости, однако каждая отдельная итерация обходится вычислительно дороже.

Week03

1. Q-learning и SARSA

Q-learning и SARSA – это алгоритмы обучения с подкреплением без модели (model-free), которые учат функцию $Q(s, a)$ непосредственно из опыта взаимодействия со средой. Они обновляют функцию ценности $Q(s, a)$ итеративно по базовому правилу:

$$Q(s, a) = \alpha \cdot \hat{Q}(s, a) + (1 - \alpha) \cdot Q(s, a).$$

Q-learning – это off-policy алгоритм, который учится находить оптимальную политику независимо от того, какие действия выполняет агент.

Правило обновления: При расчёте будущей выгоды он использует оператор максимума по всем возможным следующим действиям:

$$\hat{Q}(s, a) = r + \gamma \max_{a'} Q(s', a')$$

SARSA (State-Action-Reward-State-Action) – это on-policy алгоритм, который учится оценивать ценность политики, которой он следует.

Правило обновления: В отличие от Q-learning, SARSA использует значение Q для реально выбранного следующего действия a' , согласно текущей стратегии:

$$\hat{Q}(s, a) = r + \gamma Q(s', a')$$

2. Exploration vs Exploitation

Exploration (исследование) против Exploitation (использование) – фундаментальная дилемма в обучении с подкреплением: использовать уже известную информацию для получения максимальной награды (exploitation) или исследовать среду для поиска ещё более эффективных стратегий (exploration).

Exploitation: выбор действий, которые, согласно текущему знанию, дают наибольшую награду. Проблема заключается в том, что агент может застрять в локальном оптимуме и не обнаружить глобально оптимальную стратегию.

Exploration: выбор новых, неизведанных действий для сбора информации о среде. Проблема следующая: агент тратит время на неоптимальные действия.

Основные стратегии баланса:

1. ϵ -greedy (эпсилон-жадная стратегия): с вероятностью ϵ агент выбирает случайное действие, иначе – оптимальное. Для сходимости к оптимальной стратегии значение ϵ обычно постепенно уменьшают со временем.
2. Softmax: агент выбирает действие вероятностно, основываясь на нормализованных значениях Q . Вероятность выбора действия пропорциональна его оценке: чем выше ценность действия, тем чаще оно выбирается, но даже у плохих действий остается шанс на проверку:
$$\pi(a|s) = \text{softmax}\left(\frac{Q(s,a)}{\tau}\right),$$
 где τ – температура. При $\tau \rightarrow 0$ приближается к жадной политике.

Week04

1. DQN (Deep Q-Network)

DQN – это алгоритм, объединяющий Q-learning с глубокими нейронными сетями для аппроксимации функции $Q(s, a)$. Позволяет решать задачи с большими или непрерывными пространствами состояний, где невозможно хранить значения для каждой комбинации "состояние-действие".

Наиболее эффективная архитектура DQN принимает на вход только состояние (например, изображение экрана) и за один проход выдает вектор Q-значений для всех возможных действий в этом состоянии.

Обучение DQN рассматривается как задача регрессии:

- Целевое значение (Target) вычисляется на основе уравнения оптимальности Беллмана: $\hat{Q}(s_t, a_t) = r + \gamma \cdot \max_{a'} Q(s_{t+1}, a')$.
- Функция потерь (Loss): агент минимизирует среднеквадратичную ошибку (MSE) между текущим предсказанием сети и целевым значением: $L = (Q(s_t, a_t) - \hat{Q}(s_t, a_t))^2$.
- При обновлении весов методом градиентного спуска целевая часть уравнения считается константой.

Прямое применение нейросетей в RL нестабильно, так как тренировочные данные не являются независимыми (последовательные шаги сильно коррелируют). Для решения этой проблемы в DQN используются Experience Replay и Target Network.

2. Experience Replay

Experience Replay (воспроизведение опыта) – техника для улучшения стабильности и эффективности обучения DQN. Идея заключается в том, чтобы хранить прошлые взаимодействия агента со средой в виде кортежей (s, a, r, s') в специальном буфере (replay buffer) и обучаться на случайных выборках из этого буфера, а не только на последнем переходе. Преимущества такой техники:

1. Случайная выборка уменьшает корреляцию между последовательными обучающими примерами.
2. Каждый переход может использоваться многократно, повышая эффективность.

Данный метод применим только к off-policy алгоритмам (например, Q-learning), но не к on-policy (например, SARSA).

3. Target Network

Target Network (целевая сеть) – техника для стабилизации обучения DQN путём разделения текущей и целевой Q-функции. Проблема заключается в том, что в стандартном Q-learning целевое значение $y = r + \gamma \cdot \max_{a'} Q(s', a'; \Theta)$ зависит от тех же параметров Θ , что и оцениваемая $Q(s, a; \Theta)$. Это приводит к нестабильности: цель "движется" во время обучения.

Идея Target Network заключается в использовании отдельной сети с замороженными весами (Θ^-) для вычисления целевого значения.

- Основная сеть (Θ) обучается и обновляется на каждом шаге.
- Целевая сеть (Θ^-) используется только для расчета будущей выгоды в формуле Беллмана. Обновление целевой сети происходит либо раз в N шагов (hard update): веса целевой сети копируются из основной сети, либо очень плавно на каждом шаге (soft update) при помощи экспоненциального сглаживания: $\Theta^- = (1 - \alpha)\Theta^- + \alpha\Theta$, где α – очень малое число.

4. Double DQN

Double DQN – улучшение DQN, устраняющее завышение Q-значений. Проблема стандартного DQN в том, что он склонен завышать Q-значения из-за использования $\max_{a'} Q(s', a')$ как в выборе действия, так и в оценке его ценности (overestimation bias). Double DQN исправляет это, разделяя выбор лучшего действия и его оценку между двумя сетями:

- Действие выбирается с помощью основной сети: $a^* = \operatorname{argmax}_{a'} Q(s', a'; \Theta)$
- Целевая сеть оценивает ценность этого выбранного действия: $Q(s', a^*; \Theta^-)$

Формула целевого значения принимает вид:

$$Y_{ddqn} = r + \gamma Q(s', \operatorname{argmax}_{a'} Q(s', a'; \Theta); \Theta^-).$$

Такой подход позволяет избежать накопления ошибок и значительно повышает качество обучения агента.

Week06

1. Policy Gradient

Policy Gradient – семейство методов обучения с подкреплением, которые оптимизируют политику $\pi_{\theta}(a|s)$ напрямую, без необходимости вычисления Q-функции (в отличие от Q-learning или SARSA).

Мотивация перехода к таким методам проистекает из того, что во многих задачах вычислять точные Q-значения гораздо сложнее, чем просто выбрать оптимальное действие. Более того, в задачах с непрерывным пространством действий поиск максимума по бесконечному количеству действий в $\argmax$ становится вычислительно невозможным, тогда как в методах градиента стратегии достаточно просто использовать подходящую формулу распределения, например, нормальное распределение.

Основная идея заключается в максимизации функции $J(\theta)$ – ожидаемой награды (или дисконтированной награды) при следовании текущей стратегии. Сложность заключается в том, что параметры стратегии скрыты внутри распределения, по которому рассчитывается математическое ожидание, по которому происходит выбор действий. Для вычисления градиента ∇J используется log-derivative trick: математическое тождество $\nabla \pi(z) = \pi \nabla \log \pi(z)$. Это позволяет преобразовать аналитический градиент интеграла в форму математического ожидания:

$$\nabla J = \mathbb{E}_{s,a}[\nabla \log \pi_{\theta}(a|s) \cdot Q(s,a)].$$

Благодаря этому трюку можно оценивать градиент, просто проигрывая игровые сессии и собирая статистику.

2. REINFORCE

REINFORCE – это классический алгоритм обучения с подкреплением, относящийся к категории методов градиента стратегии (Policy Gradient). В отличие от методов, основанных на ценности (Value-based), таких как Q-learning, REINFORCE не пытается вычислить точную ценность каждого действия в таблице, а напрямую оптимизирует параметры стратегии агента $\pi_{\theta}(a|s)$, чтобы максимизировать ожидаемую награду.

Алгоритм:

1. Инициализировать веса θ случайно.
2. Сэмплировать N сессий z при текущей политике $\pi_{\theta}(a|s)$.

3. Вычислить градиент: $\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s,a \in z_i} \nabla \log \pi_0(a|s) \cdot Q(s, a)$.
4. Обновить веса (Gradient Ascend): $\Theta_{i+1} = \Theta_i + \alpha \cdot \nabla J$.

Для снижения дисперсии из оценки вычитают базовую линию $b(s)$ (baseline), которая не зависит от действия a . Это не меняет направление градиента, но стабилизирует обучение:

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^N \sum_{s,a \in z_i} \nabla \log \pi_0(a|s) \cdot (Q(s, a) - b(s))$$

Часто в качестве значения baseline выбирают функцию ценности состояния: $b(s) = V(s)$.

3. Actor-Critic

Actor-Critic – это комбинация подходов, объединяющая преимущества методов на основе политик (Policy-based) и методов на основе ценностей (Value-based). Компоненты:

- Actor (Актёр): обучает политику $\pi_\Theta(a|s)$; она отвечает за то, какие действия совершать.
- Critic (Критик): обучает функцию ценности $V_\Theta(s)$, которая оценивает действия.

Advantage Actor-Critic (A2C): в этой архитектуре критик помогает актёру обучаться быстрее, предоставляя более стабильную оценку вместо шумной полной награды G_t . Ключевым понятием здесь является Advantage (преимущество) – $A(s, a) = Q(s, a) - V(s)$, которое показывает, насколько действие a лучше среднего действия в данном состоянии. Используя уравнение Беллмана, преимущество можно оценить через временную разность: $A(s, a) = r + \gamma \cdot V(s') - V(s)$, где $V(s')$ – оценка следующего состояния критиком.

В ходе каждой итерации обновляются обе сети:

- Actor обновляется с использованием формулы градиента стратегии, где вместо Q или G подставляется значение преимущества $A(s, a)$, полученное от критика: $\Theta = \Theta + \alpha \cdot \nabla_\Theta \log \pi_\Theta(a|s) \cdot A(s, a)$
- Critic обучается минимизировать ошибку прогноза (MSE) между предсказанным $V(s)$ и реально полученной целью $r + \gamma V(s')$.

Week07

1. Применение RNN для генерации последовательностей

Рекуррентные нейронные сети (RNN) являются фундаментальным инструментом для задач, где и входные, и выходные данные представляют собой последовательности. Основной архитектурной парадигмой здесь выступает схема Encoder-Decoder (Кодировщик-Декодировщик).

Архитектура Encoder-Decoder:

- Encoder: считывает входные данные, которые могут быть последовательностью (текст на одном языке) или произвольными данными (изображение). Его задача – сжать входную информацию в скрытое состояние (hidden state).
- Decoder: генерирует выходную последовательность на основе контекстного представления. На каждом шаге декодировщик учитывает ранее сгенерированные слова и текущее скрытое состояние, чтобы предсказать следующее наиболее вероятное слово.

Примеры задач: машинный перевод, генерация подписей к изображениям, диалоговые системы.

Обычно такие модели обучаются с учителем (Supervised Learning) на больших наборах пар «вход-выход» (например, оригинал и перевод). Цель обучения – максимизировать логарифм правдоподобия (log-likelihood) правильного ответа из обучающей выборки. Однако у этого подхода есть существенный недостаток – сдвиг распределения (distribution shift): во время обучения модель видит идеальные эталонные слова, а при реальной генерации она опирается на свои собственные (возможно, ошибочные) предыдущие предсказания, что приводит к накоплению ошибок.

2. Применение RL к дообучению моделей генерации последовательностей

Обучение с подкреплением (RL) используется для преодоления ограничений стандартного обучения с учителем, позволяя модели оптимизировать недифференцируемые метрики (например, CIDEr или BLEU) и избавляться от проблемы сдвига распределения. В отличие от обучения с учителем, RL-агент обучается на своих собственных предсказаниях, а не на эталонных данных. Это позволяет модели «видеть» последствия своих ошибок и учиться их исправлять в процессе генерации последовательности. Награда (R) в данном случае выступает

как "черный ящик", оценивающий качество всей сгенерированной последовательности целиком.

Self-Critical Sequence Training (SCST). Метод, использующий REINFORCE с baseline (базовая линия). Модель генерирует две последовательности: одну жадно (выбирая самое вероятное слово), а вторую – сэмплированием. Если сэмплированная последовательность получает награду выше, чем жадная, вероятность действий из неё увеличивается. Это значительно снижает дисперсию градиента и делает обучение стабильнее.

$$A(s, a) = R(s, a) - R(s, a_{inference}(s))$$

RLHF (Reinforcement Learning from Human Feedback). Этот процесс, используемый в InstructGPT/ChatGPT, состоит из трёх этапов:

- SFT (Supervised Fine-tuning): Предварительное дообучение модели на инструкциях, написанных людьми.
- Reward Model (Модель награды): Обучение отдельной нейросети предсказывать человеческую оценку. Люди ранжируют несколько вариантов ответов модели от лучшего к худшему, и на этих данных происходит обучение.
- PPO (Proximal Policy Optimization): Основная модель генерирует ответы, получает за них оценку от модели награды и обновляет свои веса, чтобы чаще выдавать высоко оцениваемые результаты.

Direct Preference Optimization (DPO). Альтернатива RLHF, более современный и простой подход, который исключает необходимость обучения отдельной модели награды. DPO использует математическую связь между оптимальной стратегией и функцией награды, позволяя оптимизировать модель напрямую на данных о предпочтениях пользователей (какой из двух ответов лучше).